The `accumulate` function takes a sequence of numbers and accumulates the values to a single value. Suppose Agtron wants to know the total stock count of all the products on his site. The `Product` class had a list of instances:

```
class Product
  instances = []
  constructor: (stock) ->
    instances.push @
    @stock = stock
```

You want to use an accumulator to add up all the stock, but the variable `instances` is behind the scope of the `Product` class. You have to accumulate from inside `Product`:

```
class Product
  instances = []
  constructor: (stock) ->
    instances.push @
    @stock = stock
  stock: ->
    stockAccumulator = (acc, current) -> acc + current.stock
    accumulate(0, instances, stockAccumulator)
```

The `Product` class now has a reference to an `accumulate` variable defined somewhere else that isn't really a core part of `Product`. If instead you create an `enumerable` mixin, then you'll have a cleaner solution:

```
enumerable =
  accumulate: (initial=0, accumulator, sequence) ->
    total = initial
    for element in sequence
      total = accumulator total, element
    total
```

The enumerable mixin is just an object containing the accumulate function as a property.

Using this `enumerable` mixin, you can get the stock total: